



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Día, Fecha:

Viernes 20/02/2026

Hora de inicio:

17:20

0774 – Sistemas de Bases de Datos 1 Sección “N” LABORATORIO

Heinz Ricardo Gómez Galindo

Normalización Parte 1

Sistemas de Bases de Datos 1

Escuela de Ingeniería de Ciencias Y Sistemas

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Agenda

Primera Forma
Normal

Segunda Forma
Normal

Tipos de datos

Creación de tablas

Gestión de tablas

COMPETENCIA(S) QUE DESARROLLAREMOS

- Aplica conceptos de la teoría relacional convirtiendo los requerimientos funciones en tablas normalizadas con el fin de crear sistemas de almacenamiento que garanticen integridad, consistencia y eficiencia de los datos.
- Diseña modelos de bases de datos utilizando diagramas entidad-relación y normalización para garantizar integridad, consistencia y eficiencia en el acceso a los datos

Sistemas de Bases de Datos I

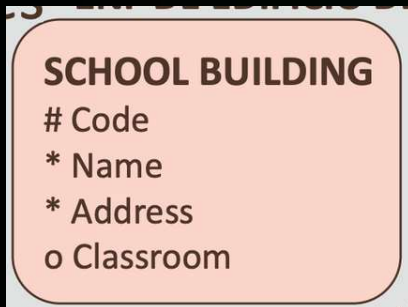
Una serie de pasos que se realizan para lograr un diseño de base de datos que permita un acceso y almacenamiento eficientes de los datos en una base de datos relacional.

Estos pasos reducen la redundancia de datos y las posibilidades de que los datos pasen a ser inconsistentes.

Redundancia: algo que es innecesariamente repetitivo

Primer Forma Normal (1NF)

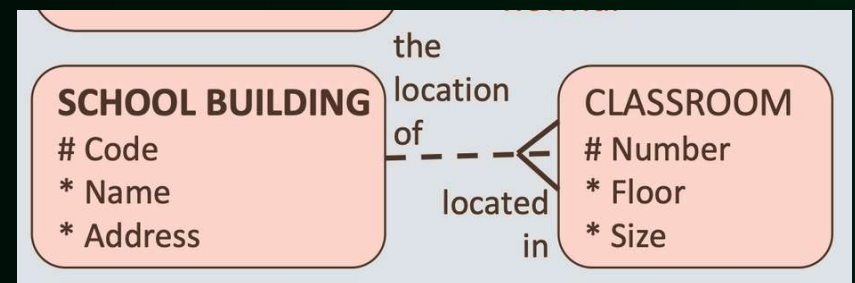
- Necesita que no existan atributos de varios valores



El atributo **Classroom** tendrá varios valores

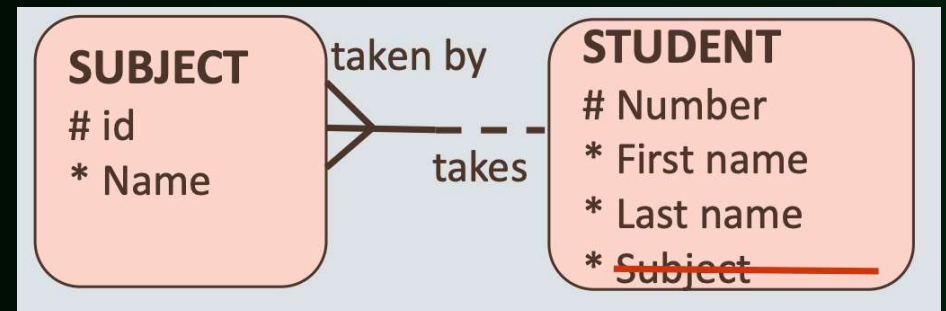
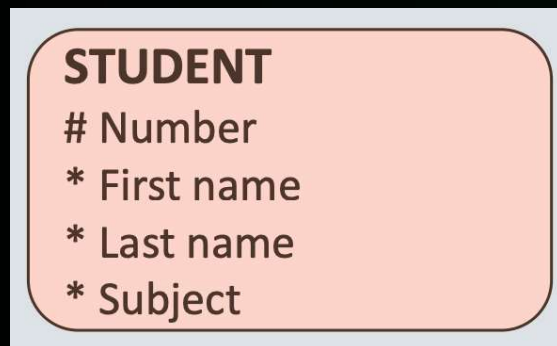
Esta entidad no está en el primer formato normal

Si un atributo tiene varios valores, creamos una entidad adicional y la relacionamos con la entidad original con una relación 1:M



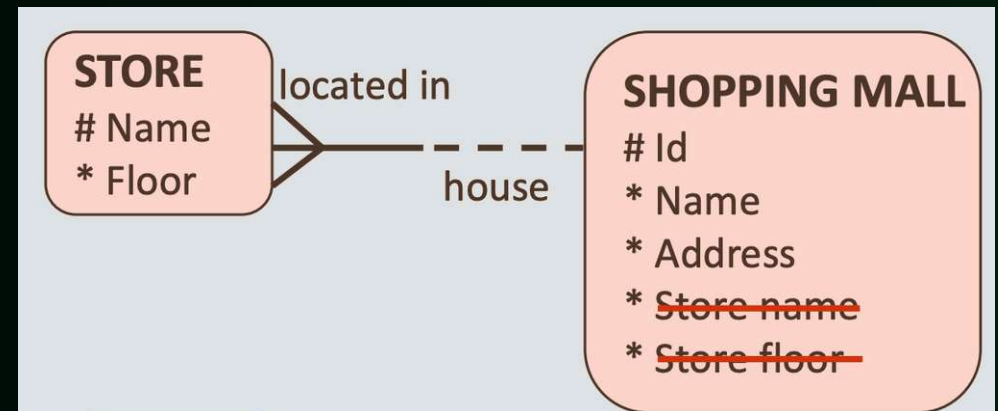
Primer Forma Normal (1NF)

Ejemplo 1



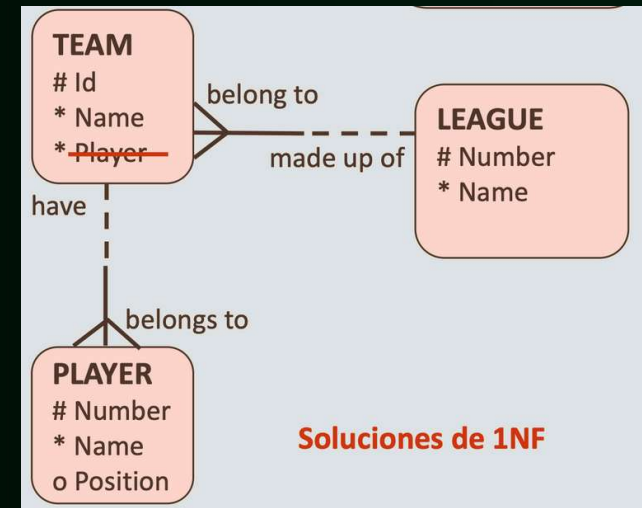
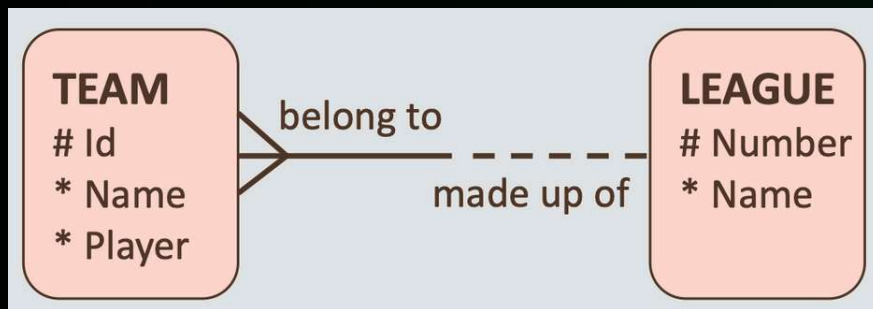
Primer Forma Normal (1NF)

Ejemplo 2



Primer Forma Normal (1NF)

Ejemplo 3



Segunda Forma Normal (1NF)

- Necesita que cualquier atributo que no sea ID dependa de todo el ID



El ID es un **ID compuesto** que consta del número de proveedor y el número de producto

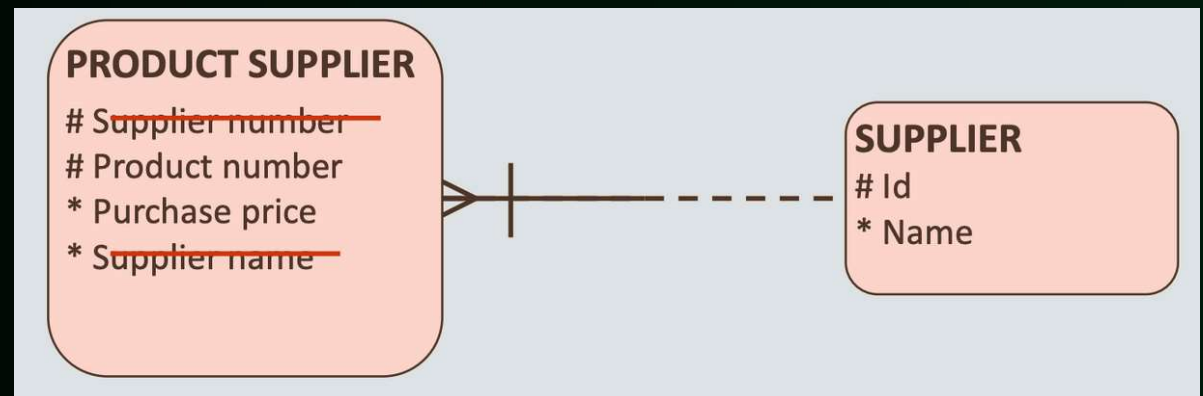
Si un proveedor proporciona 5 productos diferentes y, a continuación, se crean 5 instancias distintas

¿Qué sucede si el nombre de proveedor cambia?

El nombre de proveedor se tendrá que cambiar en 5 instancias distintas

Segunda Forma Normal (1NF)

- Necesita que cualquier atributo que no sea ID dependa de todo el ID





ORACLE

Academy



Database Programming with SQL

13-2

Uso de Tipos de Dato

ORACLE
Academy



Visión General de Tipo de Dato

- Cada valor manipulado por Oracle tiene un tipo de dato
- El tipo de dato de un valor asocia un juego fijo de propiedades al valor
- Estas propiedades hacen que la base de datos trate los valores de un tipo de dato de forma diferente a valores de otro tipo de dato

Visión General de Tipo de Dato

- Los distintos tipos de dato ofrecen varias ventajas:
 - Las columnas de un solo tipo producen resultados consistentes
 - Por ejemplo, las columnas del tipo de dato DATE siempre producen valores de fecha
 - No se puede insertar el tipo de dato incorrecto en una columna. Por ejemplo, las columnas del tipo de dato DATE impedirán que se inserten datos de tipo NUMBER
- Por este motivo, cada columna de una base de datos relacional solo puede contener un tipo de dato
- No puede combinar los tipos de dato en una columna

Tipos de Dato Comunes

- A continuación se muestran los tipos de dato de columna más utilizados para los valores de carácter y número
- Para valores de carácter:
 - CHAR (tamaño fijo, máximo de 2000 caracteres)
 - VARCHAR2 (tamaño variable, máximo de 32,767 caracteres)
 - CLOB (tamaño variable, máximo de 128 terabytes)
- Para valores de número:
 - NUMBER (tamaño variable, precisión máxima de 38 dígitos)

Tipos de Dato Comunes

- A continuación se muestran los tipos de dato de columna más utilizados para los valores de fecha, hora y binario
- Para valores de fecha y hora:
 - DATE
 - TIMESTAMP
 - INTERVAL
- Para valores binarios (p. ej., multimedia: JPG, WAV, MP3, etc.):
 - RAW (tamaño variable, máximo de 2000 bytes)
 - BLOB (tamaño variable, máximo de 128 terabytes)

Tipos de Dato Comunes

- Para valores de caracteres, normalmente es mejor utilizar VARCHAR2 o CLOB que CHAR, porque se ahorra espacio
- Por ejemplo, el apellido de un empleado es "Chang"
- En una columna VARCHAR2(30), solo se almacenan los 5 caracteres importantes: C h a n g
- Pero en una columna CHAR(30), se almacenarán también 25 espacios finales, para formar un tamaño fijo de 30 caracteres
- Los valores de número pueden ser negativos, así como positivos. Por ejemplo, NUMBER(6,2) puede almacenar cualquier valor de +9999,99 hasta -9999,99

Tipos de Dato DATE-TIME

- El tipo de dato DATE almacena un valor de siglos hasta segundos enteros, pero no se pueden almacenar las fracciones de un segundo
- "21-Aug-2003 17:25:30" es un valor válido, pero "21-Aug-2003 17:25:30.255" no lo es
- El tipo de dato TIMESTAMP es una extensión del tipo de dato DATE que permite fracciones de un segundo
- Por ejemplo, TIMESTAMP(3) permite 3 dígitos después de los segundos completos, lo que permite almacenar valores de hasta milisegundos

Tipos de Dato DATE-TIME

- Ejemplo de TIMESTAMP:

```
CREATE TABLE time_ex1  
(exact_time TIMESTAMP);
```

```
INSERT INTO time_ex1  
VALUES ('10-Jun-2017 10:52:29.123456');
```

```
INSERT INTO time_ex1  
VALUES (SYSDATE);
```

```
INSERT INTO time_ex1  
VALUES (SYSTIMESTAMP);
```

```
SELECT *  
FROM time_ex1;
```

EXACT_TIME
10-JUN-15 10.52.29.123456 AM
16-JUL-15 08.17.08.000000 AM
16-JUL-15 08.17.16.610293 AM

TIMESTAMP...With [LOCAL] Time Zone

- Piense en el valor de hora "17:30". Por supuesto, significa "cinco y media de la tarde"
- ¿Pero en qué zona horaria?
- ¿Es cinco y media de la hora de Nueva York, de Pekín, de Estambul tiempo... ?
- En las organizaciones globalizadas actuales que operan en muchos países distintos, es importante saber a qué zona horaria hacer referencia un valor de fecha-hora

TIMESTAMP...With [LOCAL] Time Zone

- `TIMESTAMP WITH TIME ZONE` almacena un valor de zona horaria como un desplazamiento de la Hora Universal Coordinada o UCT (también conocida como Hora Media de Greenwich o GMT)
- Un valor "21-Aug-2003 08:00:00 -5:00" significa 8:00 a.m. 5 horas por debajo de UTC
- Esta es Hora Oficial Oriental de EE. UU. (EST)

TIMESTAMP...With [LOCAL] Time Zone

- Ejemplo de TIMESTAMP WITH TIME ZONE:

```
CREATE TABLE time_ex2  
(time_with_offset TIMESTAMP WITH TIME ZONE);
```

```
INSERT INTO time_ex2  
VALUES (SYSTIMESTAMP);
```

```
INSERT INTO time_ex2  
VALUES ('10-Jun-2017 10:52:29.123456 AM +2:00');
```

```
SELECT *  
FROM time_ex2;
```

TIME_WITH_OFFSET
16-JUL-15 08.49.47.126056 AM -07:00
10-JUN-15 10.52.29.123456 AM +02:00

TIMESTAMP...With [LOCAL] Time Zone

- **TIMESTAMP WITH LOCAL TIME ZONE** es similar, pero con una diferencia: si esta columna se selecciona en una sentencia SQL, la hora se convierte automáticamente a la zona horaria del usuario que realiza la selección
- Ejemplo de **TIMESTAMP With...Time Zone**:

```
CREATE TABLE time_ex3
( first_column TIMESTAMP WITH TIME ZONE,
  second_column TIMESTAMP WITH LOCAL TIME ZONE);
```

```
INSERT INTO time_ex3
  (first_column, second_column)
VALUES
  ('15-Jul-2017 08:00:00 AM -07:00', '15-Nov-2007 08:00:00');
```

Ejemplo de TIMESTAMP...With Time Zone

- Ambos valores se almacenan con un desplazamiento de hora de -07:00 horas (PST)
- Pero ahora un usuario en Estambul ejecuta:

```
SELECT *  
FROM time_ex3;
```

FIRST_COLUMN	SECOND_COLUMN
15-JUL-15 08.00.00.000000 AM -07:00	15-NOV-07 05.00.00.000000 PM

- La hora de Estambul es de 9 horas por delante de PST; cuando son las 8:00 a.m. en Los Ángeles, son las 5:00 p.m. en Estambul

Tipos de Dato INTERVAL

- Estos almacenan el tiempo o intervalo de tiempo transcurrido entre dos valores de fecha y hora
- INTERVAL YEAR TO MONTH almacena un período de tiempo medido en años y meses
- INTERVAL DAY TO SECOND almacena un período de tiempo medido en días, horas, minutos y segundos



INTERVAL YEAR...TO MONTH

- Sintaxis:

```
INTERVAL YEAR [(year_precision)] TO MONTH
```

- year_precision es el número máximo de dígitos del elemento YEAR
- El valor por defecto de year_precision es 2



INTERVAL YEAR...TO MONTH

- En este ejemplo se muestra INTERVAL YEAR TO

MONTH:

```
CREATE TABLE time_ex4
(loan_duration1 INTERVAL YEAR(3) TO MONTH,
 loan_duration2 INTERVAL YEAR(2) TO MONTH);
```

```
INSERT INTO time_ex4 (loan_duration1, loan_duration2)
VALUES (INTERVAL '120' MONTH(3),
        INTERVAL '3-6' YEAR TO MONTH);
```

Suponga que la fecha de hoy es: 17-Jul-2017

```
SELECT SYSDATE + loan_duration1 AS "120 months from now",
       SYSDATE + loan_duration2 AS "3 years 6 months from
now"
FROM time_ex4;
```

120 months from now (120 meses a partir de ahora)	3 years 6 months from now (3 años y 6 meses a partir de ahora)
17-Jul-2027	17-Jan-2021

INTERVAL DAY...TO SECOND

- Utilice esta opción cuando necesite una diferencia más precisa entre dos valores de fecha-hora
- Sintaxis:

```
INTERVAL DAY [day_precision] TO SECOND  
[(fractional_seconds_precision)]
```

- Day_precision es el número máximo de dígitos del elemento DAY
- El valor por defecto de day_precision es 2
- fractional_seconds_precision es el número de dígitos de la parte fraccionaria del campo de fecha/hora SECOND
- El valor por defecto es 6

INTERVAL DAY...TO SECOND

- En este ejemplo se muestra INTERVAL DAY TO SECOND:

```
CREATE TABLE time_ex5  
(day_duration1 INTERVAL DAY(3) TO SECOND,  
 day_duration2 INTERVAL DAY(3) TO SECOND);
```

```
CREATE TABLE time_ex5  
(day_duration1 INTERVAL DAY(3) TO SECOND,  
 day_duration2 INTERVAL DAY(3) TO SECOND);
```

```
SELECT SYSDATE + day_duration1 AS "25 Days from now",  
       TO_CHAR(SYSDATE + day_duration2, 'dd-Mon-yyyy hh:mi:ss')  
       AS "precise days and time from now"  
FROM time_ex5;
```

25 Days from now (25 días a partir de ahora)	precise days and time from now (Días y hora precisos a partir de ahora)
11-Aug-2017	21-Jul-2017 01:13:17



ORACLE

Academy



Database Programming with SQL

13-1

Creación de tablas

ORACLE
Academy



Objetos de Esquema de la Base de Datos

- Algunos de los tipos de objetos ocupan espacio, lo que se denomina almacenamiento, en la base de datos y otros no
- Los objetos de base de datos que ocupan mucho espacio de almacenamiento se denominan segmentos
- Las tablas y los índices son ejemplos de segmentos, ya que los valores almacenados en las columnas de cada fila ocupan mucho espacio en el disco físico
- Las vistas, restricciones, secuencias y sinónimos también son objetos, pero el único espacio que necesitan en la base de datos está en la definición del objeto: ninguno de ellos tiene filas de datos asociadas

Objetos de Esquema de la Base de Datos

- La base de datos almacena las definiciones de todos los objetos de base de datos en el diccionario de datos y estas definiciones están accesibles para todos los usuarios de la base de datos así como para la base de datos en sí
- ¿Se ha preguntado alguna vez cómo sabe Oracle qué columnas devolver de una consulta?
- Por ejemplo, si especifica `SELECT * FROM jobs` en lugar de `SELECT job_id, job_title FROM jobs`, ¿cómo sabe Oracle qué columnas devolver?
- La base de datos busca la definición de la tabla utilizada en la consulta, traduce el "*" en la lista completa de columnas, y devuelve el resultado

Objetos de Esquema de la Base de Datos

- La base de datos utiliza el diccionario de datos para todas las sentencias que emita, incluso aunque enumere los nombres de columna en lugar de utilizar "*"
- Comprueba que las tablas a las que hace referencia en la sentencia existen en la base de datos, comprueba que los nombres de columna son correctos, comprueba si tiene los privilegios correctos para realizar la acción que está solicitando y, finalmente, utiliza el diccionario de datos para decidir el plan de ejecución, cómo realizará en realidad la solicitud

Objetos de Esquema de la Base de Datos

- Mediante la ventana SQL Commands, debe solicitar una **DESCRipción de la tabla**
- Ninguna de las opciones adicionales que ofrece Object Browser está disponible en esta interfaz

```
DESCRIBE jobs;
```

Results Explain Describe Saved SQL History									
Object Type		TABLE ?		Object		JOBS ?			
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
JOBS	JOB_ID	VARCHAR2	10	-	-	1	-	-	-
	JOB_TITLE	VARCHAR2	35	-	-	-	-	-	-
	MIN_SALARY	NUMBER	-	6	0	-	✓	-	-
	MAX_SALARY	NUMBER	-	6	0	-	✓	-	-

Creación de Tabla

- Todos los datos de una base de datos relacional se almacenan en tablas
- Al crear una nueva tabla, utilice las siguientes reglas para los nombres de tabla y columnas:
 - Deben empezar por una letra
 - Deben tener entre 1 y 30 caracteres
 - Solo deben contener A - Z, a - z, 0 - 9, _ (carácter de subrayado), \$ y #
 - No deben ser un duplicado de otro nombre de objeto propiedad del mismo usuario
 - No deben ser una palabra reservada de Oracle Server

Reglas de Nomenclatura

- Es mejor utilizar nombres descriptivos para las tablas y otros objetos de la base de datos
- Si una tabla va a almacenar información sobre los alumnos, asígnele el nombre ALUMNOS, no PERSONAS o NIÑOS
- Los nombres de tablas no son sensibles a mayúsculas/minúsculas
- Por ejemplo, ALUMNOS se trata igual que ALuMnoS o alumnos



Reglas de Nomenclatura

- Los nombres de tabla deben ser en plural, por ejemplo, ALUMNOS, no alumno
- La creación de tablas forma parte del lenguaje de definición de datos (DDL) de SQL
- En las sentencias DDL que se utilizan para configurar, cambiar y eliminar estructuras de datos de tablas se incluyen ALTER, DROP, RENAME y TRUNCATE



CREATE TABLE

- Para crear una nueva tabla, debe tener el privilegio **CREATE TABLE** y un área de almacenamiento para él
- El administrador de la base datos utiliza sentencias de lenguaje de control de datos (DCL) para otorgar este privilegio a los usuarios y asignar un área de almacenamiento
- Las tablas que pertenecen a otros usuarios no se incluyen en su esquema
- Si desea utilizar una tabla que no está en su esquema, utilice el nombre del propietario de la tabla como prefijo para el nombre de tabla:

```
SELECT *  
FROM mary.students;
```

Sintaxis de CREATE TABLE

- Para crear una nueva tabla, utilice los siguientes detalles de sintaxis:
 - table es el nombre de la tabla
 - column es el nombre de la columna
 - datatype es el tipo de dato y la longitud de la columna
 - La expresión DEFAULT especifica un valor por defecto si se omite un valor en la sentencia INSERT

```
CREATE TABLE table  
(column data type [DEFAULT expression],  
column data type [DEFAULT expression],  
(..... [ ] ) ;
```

Ejemplo de CREATE TABLE

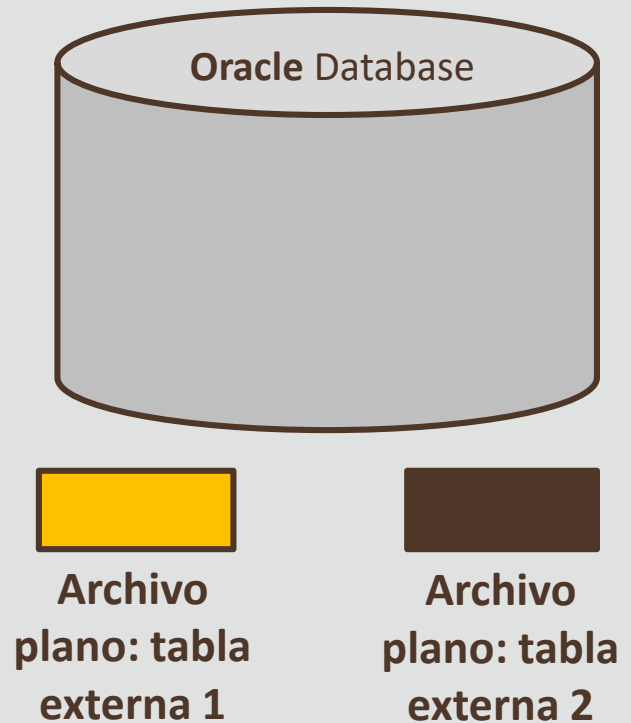
- Los ejemplos siguientes muestran la sentencia CREATE TABLE:

```
CREATE TABLE my_cd_collection
(cd_number NUMBER(3),
 title VARCHAR2(20),
 artist VARCHAR2(20),
 purchase_date DATE DEFAULT SYSDATE);
```

```
CREATE TABLE my_friends
(first_name VARCHAR2(20),
 last_name VARCHAR2(30),
 email VARCHAR2(30),
 phone_num VARCHAR2(12),
 birth_date DATE);
```

Tablas Externas

- Oracle también soporta otro tipo de tabla: tabla externa
- En una tabla externa, las filas de datos no están en los archivos de la base de datos, sino que en su lugar se encuentran en un archivo plano, almacenado de forma externa a la base de datos
- Normalmente, una tabla externa se utiliza para almacenar datos migrados de versiones anteriores de las bases de datos utilizadas por una compañía



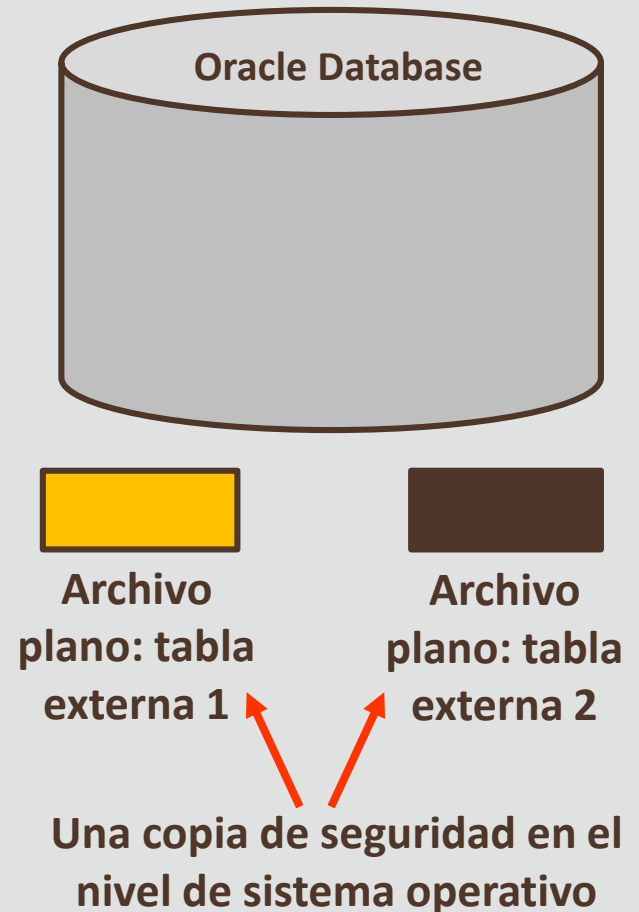
Tablas Externas

- Cuando una compañía implanta una nueva aplicación y base de datos, suele tener que importar la mayoría de los datos de los sistemas antiguos al nuevo sistema para el acceso de lectura y escritura normal, pero puede haber algunos datos que no se utilicen con frecuencia y, por lo tanto, solo se tenga que acceder a ellos para lectura
- Este tipo de dato podría alojarse en una tabla externa



Tablas Externas

- Una de las principales ventajas de Oracle es que solo se tiene que hacer una copia de seguridad una vez de los datos alojados en tablas externas y, a continuación, nunca más a menos que el contenido del archivo cambie
- La sintaxis para crear una tabla externa es muy similar a la de crear una tabla estándar, excepto por el hecho de que tiene sintaxis adicional al final



Tablas Externas

- La nueva sintaxis (que aparece en rojo) de las siguientes diapositivas no se utiliza en sentencias SQL estándar para la creación de tablas
- ORGANIZATION EXTERNAL: indica a Oracle que cree una tabla externa
- TYPE ORACLE_LOADER: de tipo Oracle Loader (producto de Oracle)
- DEFAULT DIRECTORY def_dir1: nombre del directorio para el archivo

Tablas Externas

- ACCESS PARAMETERS: cómo leer el archivo
- RECORDS DELIMITED BY NEWLINE: cómo identificar el principio de una nueva fila
- FIELDS: especificaciones de nombre de campo y tipo de dato
- LOCATION: nombre del archivo real que contiene los datos
- Un ejemplo de la nueva sintaxis se muestra en rojo en la siguiente diapositiva

Ejemplo de Tablas Externas

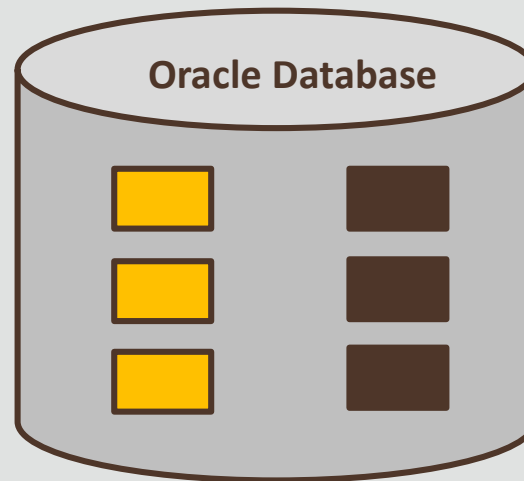
```
CREATE TABLE emp_load
  (employee_number CHAR(5),
   employee_dob CHAR(20),
   employee_last_name CHAR(20),
   employee_first_name CHAR(15),
   employee_middle_name CHAR(15),
   employee_hire_date DATE)
ORGANIZATION EXTERNAL
  (TYPE ORACLE_LOADER
   DEFAULT DIRECTORY def_dir1
   ACCESS PARAMETERS
   (RECORDS DELIMITED BY NEWLINE
    FIELDS (employee_number CHAR(2),
           employee_dob CHAR(20),
           employee_last_name CHAR(18),
           employee_first_name CHAR(11),
           employee_middle_name CHAR(11),
           employee_hire_date CHAR(10) date_format DATE mask
"mm/dd/yyyy"))
   LOCATION ('info.dat'));
```

ORACLE

Academy

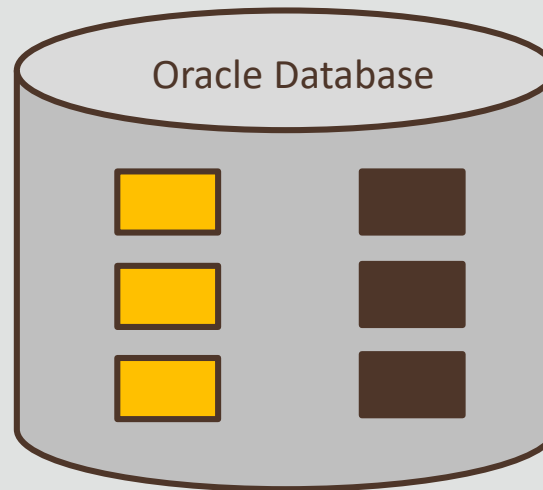
Data Dictionary

- Existen dos tipos de tablas en una base de datos
Oracle: tablas de usuario y tablas de diccionario de datos
- Se pueden emitir sentencias SQL para acceder a ambos tipos de tablas: puede seleccionar, insertar, actualizar y suprimir datos de las tablas de usuario y puede seleccionar datos de la tabla de diccionario de datos



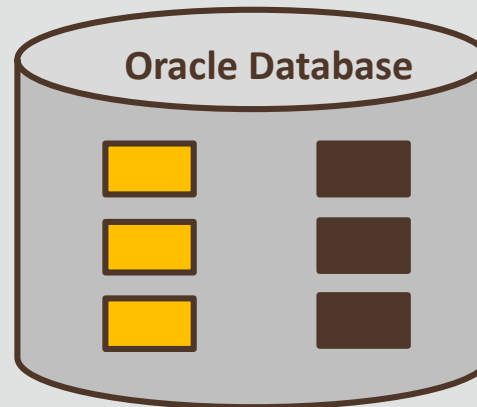
Data Dictionary

- Las tablas de usuario creadas por el usuario que contiene sus datos:
 - empleados, departamentos, cargos, etc.
- Tablas de diccionario de datos:
 - DICTIONARY, USER_OBJECTS, USER_TABLES, USER_SEGMENTS, USER_INDEXES, etc.



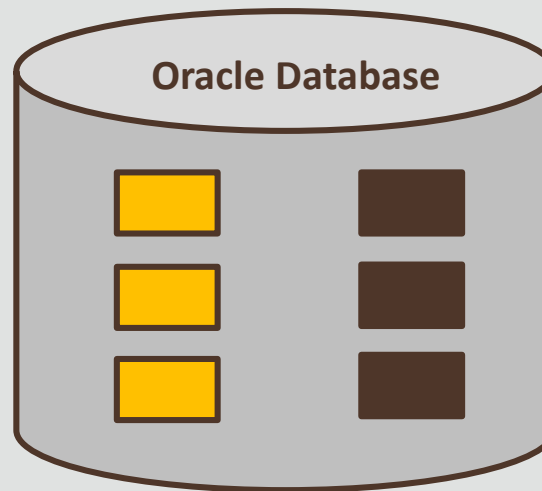
Data Dictionary

- Las tablas de diccionario de datos son propiedad de un usuario de Oracle especial llamado SYS y solo se deben utilizar sentencias SELECT al trabajar con cualquiera de estas tablas
- Para proteger a estas tablas del acceso accidental de usuario, todas tienen vistas creadas a través de las que los usuarios de la base de datos acceden al diccionario de datos



Data Dictionary

- Si cualquier usuario de Oracle intenta realizar inserciones, actualizaciones o supresiones en una de las tablas de diccionario de datos, no se permite la operación ya que puede poner en peligro la integridad de toda la base de datos



Data Dictionary

- Al utilizar las vistas del diccionario de datos en la interfaz SQL Commands, debe saber los nombres de las vistas del diccionario con las que está trabajando
- En Oracle, esto es bastante sencillo: ponga como prefijo al tipo de objeto que está buscando USER_xxx o ALL_xxx, donde xxx es el tipo de objeto

```
SELECT table_name, status  
FROM USER_TABLES;
```

```
SELECT table_name, status  
FROM ALL_TABLES;
```

Data Dictionary

- Por lo tanto, si desea investigar los índices, simplemente seleccione USER_INDEXES; si desea información acerca de las secuencias, la tabla es USER_SEQUENCES, etc.

```
SELECT *  
FROM user_indexes;
```

```
SELECT *  
FROM user_objects  
WHERE object_type = 'SEQUENCE';
```



ORACLE

Academy



Database Programming with SQL

13-3

Modificación de una Tabla

ORACLE
Academy



ALTER TABLE

- Las sentencias ALTER TABLE se utilizan para:
 - Agregar una nueva columna
 - Modificar una columna existente
 - Definir un valor DEFAULT para una columna
 - Borrar una columna
- Puede agregar o modificar una columna de una tabla, pero no puede especificar donde aparece la columna

ALTER TABLE

- Una columna recién agregada siempre se convierte en la última columna de la tabla
- Además, si una tabla ya tiene filas de datos y agrega una nueva columna a la tabla, la nueva columna inicialmente será nula para todas las filas ya existentes



ALTER TABLE: Adición de una Columna

- Para agregar una nueva columna, utilice la sintaxis SQL que se muestra a continuación:

```
ALTER TABLE tablename  
ADD (column name data type [DEFAULT expression],  
column name data type [DEFAULT expression], ...
```

- Por ejemplo:

```
ALTER TABLE my_cd_collection  
ADD (release_date DATE DEFAULT SYSDATE);
```

```
ALTER TABLE my_friends  
ADD (favorite_game VARCHAR2(30));
```

ALTER TABLE: Modificación de una Columna

- La modificación de una columna puede incluir cambios en el tipo de dato, tamaño y valor DEFAULT de una columna
- Las reglas y las restricciones al modificar una columna son las siguientes:
 - Puede aumentar el ancho o la precisión de una columna numérica
 - Puede aumentar el ancho de una columna de caracteres
 - Puede reducir el ancho de una columna NUMBER si la columna contiene únicamente valores nulos o si la tabla no tiene filas
 - Para tipos VARCHAR, puede reducir el ancho hasta el mayor valor incluido en la columna

ALTER TABLE: Modificación de una Columna

- Puede cambiar el tipo de dato solo si la columna contiene valores nulos
- Solo puede convertir una columna CHAR a VARCHAR2 o una columna VARCHAR2 a CHAR si la columna contiene valores nulos, o bien si no cambia el tamaño a algo más pequeño que cualquier valor de la columna
- El cambio del valor DEFAULT de una columna solo afecta a las inserciones posteriores en la tabla

Ejemplo de ALTER TABLE: Modificación de una Columna

- Ejemplo: se ha creado una tabla con dos columnas:

```
CREATE TABLE mod_emp  
  (last_name VARCHAR2(20) ,  
   salary NUMBER(8,2)) ;
```

- ¿Cuál de estas modificaciones estaría permitida y cuál no?
- Considere las respuestas con y sin filas de datos en la tabla

```
ALTER TABLE mod_emp  
  MODIFY (last_name VARCHAR2(30)) ;
```

```
ALTER TABLE mod_emp  
  MODIFY (last_name VARCHAR2(10)) ;
```

```
ALTER TABLE mod_emp  
  MODIFY (salary NUMBER(10,2)) ;
```

```
ALTER TABLE mod_emp  
  MODIFY (salary NUMBER(8,2) DEFAULT 50) ;
```

Ejemplo de ALTER TABLE: Modificación de una Columna

- Solo estaría permitida si las columnas estuvieran vacías o el nombre más largo tuviera 10 caracteres o menos

```
ALTER TABLE mod_emp  
    MODIFY (last_name VARCHAR2(10));
```

- Estaría permitida con o sin datos ya que aumenta el ancho de columna

```
ALTER TABLE mod_emp  
    MODIFY (last_name VARCHAR2(30));
```

Ejemplo de ALTER TABLE: Modificación de una Columna

- Estaría permitida con o sin datos ya que aumenta la precisión de columna

```
ALTER TABLE mod_emp  
    MODIFY (salary NUMBER(10,2));
```

- Se permitiría con o sin datos, ya que solo se agrega un valor DEFAULT

```
ALTER TABLE mod_emp  
    MODIFY (salary NUMBER(8,2) DEFAULT 50);
```

ALTER TABLE: Borrado de una Columna

- Al borrar una columna se aplican las siguientes reglas:
 - Se puede borrar una columna que contenga datos
 - Solo se puede borrar una columna cada vez
 - No puede borrar todas las columnas de una tabla; debe permanecer al menos una columna
 - Una vez borrada la columna, los valores de datos de esta no se pueden recuperar

ALTER TABLE: Borrado de una Columna

- Sintaxis SQL:

```
ALTER TABLE tablename  
DROP COLUMN column name;
```

- Por ejemplo:

```
ALTER TABLE my_cd_collection  
DROP COLUMN release_date;
```

```
ALTER TABLE my_friends  
DROP COLUMN favorite_game;
```

SET UNUSED para Columnas

- Borrar una columna de una tabla grande puede llevar mucho tiempo
- Una alternativa más rápida es marcar la columna como no utilizable
- Los valores de columna permanecen en la base de datos, pero no se puede acceder a ellos de ninguna forma, por lo que el efecto es el mismo que borrar la columna
- De hecho, puede agregar una nueva columna a la base de datos con el mismo nombre que la columna no utilizada
- Las columnas no utilizadas existen, pero son invisibles
- Sintaxis: `ALTER TABLE tablename SET UNUSED (column name);`

Ejemplo de SET UNUSED para Columnas

- Ejemplo:

```
ALTER TABLE copy_employees  
  SET UNUSED (email);
```

- DROP UNUSED COLUMNS elimina todas las columnas marcadas actualmente como no utilizadas
- Utilice esta sentencia cuando desee reclamar el espacio en disco adicional de las columnas no utilizadas en una tabla

- Ejemplo:

```
ALTER TABLE copy_employees  
  DROP UNUSED COLUMNS;
```

Resumen de ALTER TABLE

- En este gráfico se resumen los usos del comando ALTER TABLE:

Sintaxis	Resultados	Problemas
ALTER TABLE tablename ADD (column name data type [DEFAULT expression], column name data type [DEFAULT expression], ...	Agrega una nueva columna a la tabla	No puede especificar dónde debe aparecer la columna en la tabla. Se convierte en la última columna.
ALTER TABLE tablename MODIFY (column name data type [DEFAULT expression], column name data type, ...	Se utiliza para cambiar el tipo de dato, tamaño y valor por defecto de una columna.	El cambio del valor por defecto de una columna solo afecta a las inserciones posteriores en la tabla.
ALTER TABLE tablename DROP COLUMN column name;	Se utiliza para borrar una columna de una tabla	Debe quedar al menos una columna en la tabla después de modificarla. Una vez borrada, la columna no se puede recuperar.
ALTER TABLE tablename SET UNUSED (column name);	Se utiliza para marcar una o más columnas para que se puedan borrar más tarde	No restaura espacio en disco. Las columnas se tratan como si se hubieran borrado.
ALTER TABLE tablename DROP UNUSED COLUMNS	Elimina de la tabla todas las columnas marcadas actualmente como no utilizadas.	Una vez definidas como no utilizadas, no hay acceso a las columnas; no se muestra ningún dato con DESCRIBE. Eliminación permanente; sin rollback.

DROP TABLE

- La sentencia DROP TABLE elimina la definición de una tabla de Oracle
- La base de datos pierde todos los datos de la tabla y los índices asociados a los mismos
- Cuando se emite una sentencia DROP TABLE:
 - Se suprimen todos los datos de la tabla
 - Se elimina la descripción de la tabla del diccionario de datos
- Oracle Server no cuestiona la decisión y borra la tabla inmediatamente

DROP TABLE

- En la siguiente diapositiva, verá que puede restaurar una tabla después borrarla, pero no está garantizado
- Solo el creador de la tabla o un usuario con el privilegio DROP ANY TABLE (normalmente solo el DBA) puede eliminar una tabla
- Sintaxis:

```
DROP TABLE tablename;
```

- Ejemplo:

```
DROP TABLE copy_employees;
```

FLASHBACK TABLE

- Si borra una tabla por error, puede recuperar esa tabla y sus datos
- Cada usuario de base de datos tiene su propia papelera de reciclaje a la que se mueven los objetos borrados y que se pueden recuperar de aquí con el comando FLASHBACK TABLE
- Este comando se puede utilizar para restaurar una tabla, una vista o un índice que se haya borrado por error
- La sintaxis es la siguiente:

```
FLASHBACK TABLE tablename TO BEFORE DROP;
```

FLASHBACK TABLE

- Por ejemplo, si borra la tabla EMPLOYEES por error, puede recuperarla simplemente emitiendo el comando:

```
FLASHBACK TABLE copy_employees TO BEFORE DROP;
```

- Como propietario de una tabla, puede emitir el comando de flashback y si la tabla que está restaurando tenía índices, estos también se restauran
- Es posible ver los objetos que se pueden restaurar consultando la vista del diccionario de datos USER_RECYCLEBIN



FLASHBACK TABLE

- Se puede consultar la vista USER_RECYCLEBIN como todas las demás vistas del diccionario de datos:

```
SELECT original_name, operation, droptime  
FROM user_recyclebin
```

ORIGINAL_NAME	OPERATION	DROPTIME
EMPLOYEES	DROP	2007-12-05:12.34.24
EMP_PK	DROP	2007-12-05:12.34.24



FLASHBACK TABLE

- Una vez restaurada una tabla con el comando FLASHBACK TABLE, ya no está visible en la vista USER_RECYCLEBIN
- Cualquier índice que se borrara con la tabla original también se restaurará
- Puede que sea necesario (por motivos de seguridad) borrar una tabla completamente, omitiendo la papelera de reciclaje
- Esto se puede hacer agregando la palabra clave PURGE

```
DROP TABLE copy_employees PURGE;
```

RENAME

- Para cambiar el nombre de una tabla, utilice la sentencia RENAME
- Esto solo lo puede hacer el propietario del objeto o el DBA
- Sintaxis:

```
RENAME old_name to new_name;
```

- Ejemplo:

```
RENAME my_cd_collection TO my_music;
```

- Veremos más adelante que podemos renombrar otros tipos de objetos como, por ejemplo, vistas, secuencias y sinónimos

TRUNCATE

- Al truncar una tabla, se eliminan todas las filas de una tabla y se libera el espacio de almacenamiento utilizado por dicha tabla
- Al utilizar la sentencia TRUNCATE TABLE:
 - No se puede realizar rollback de la eliminación de filas
 - Debe ser el propietario de la tabla o tener privilegios de sistema DROP ANY TABLE

TRUNCATE

- Sintaxis:

```
TRUNCATE TABLE tablename;
```

- Con la sentencia DELETE también se eliminan todas las filas de una tabla, pero no se libera el espacio de almacenamiento
- TRUNCATE es más rápido que DELETE porque no genera información de rollback

COMMENT ON TABLE

- Puede agregar un comentario de hasta 2.000 caracteres sobre una columna, tabla o vista mediante la sentencia COMMENT
- Sintaxis:

```
COMMENT ON TABLE tablename | COLUMN table.column  
IS 'place your comment here';
```

- Ejemplo:

```
COMMENT ON TABLE employees  
IS 'Western Region only';
```

COMMENT ON TABLE

- Para ver los comentarios de la tabla en el diccionario de datos:

```
SELECT table_name, comments  
FROM user_tab_comments;
```

TABLE_NAME	COMMENTS
EMPLOYEES	Western Region Only

- Si desea borrar un comentario realizado anteriormente en una tabla o columna, utilice la cadena vacía ("):

```
COMMENT ON TABLE employees IS ' ' ;
```

FLASHBACK QUERY

- Puede que detecte que los datos en una tabla se han cambiado, de algún modo, de forma incorrecta
- Afortunadamente, Oracle tiene una función que permite ver los datos de fila en momentos concretos en el tiempo, por lo que puede comparar diferentes versiones de una fila a lo largo del tiempo
- Esta función es muy útil
- Imagine, por ejemplo, que alguien por error ha realizado algún DML en una tabla y, a continuación, ejecuta COMMIT en dichos cambios
- Oracle Application Express confirma automáticamente, por lo que se cometen fácilmente errores

FLASHBACK QUERY

- Puede utilizar la función FLASHBACK QUERY para examinar qué aspecto tenían las filas ANTES de que se aplicaran esos cambios
- Cuando Oracle cambia datos, siempre mantiene una copia del aspecto de los datos corregidos antes de que se realizara cualquier cambio
- Por lo que mantiene una copia del valor anterior de la columna para una actualización de columna, mantiene toda la fila para una supresión y no mantiene nada para una sentencia de inserción

FLASHBACK QUERY

- Estas copias antiguas se mantienen en un lugar especial denominado el tablespace UNDO
- Los usuarios pueden acceder a esta zona especial de la base de datos mediante una consulta de flashback
- Puede consultar las versiones anteriores de los datos utilizando la cláusula VERSIONS en una sentencia SELECT

FLASHBACK QUERY

- Por ejemplo:

```
SELECT employee_id,first_name ||' '|| last_name AS "NAME",  
       versions_operation AS "OPERATION",  
       versions_starttime AS "START_DATE",  
       versions_endtime  AS "END_DATE", salary  
FROM employees  
     VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE  
WHERE  employee_id = 1;
```

- El número SCN al que se hacía referencia en la consulta anterior significa que el número de cambio del sistema es una identificación precisa de tiempo en la base de datos
- Se trata de un número secuencial que incrementa y mantiene la propia base de datos

FLASHBACK QUERY

- La mejor forma de demostrar FLASHBACK QUERY es con un ejemplo
- El contenido es el siguiente para employee_id 1 en la tabla de empleados

```
SELECT employee_id,first_name ||' '|| last_name AS "NAME",  
       versions_operation AS "OPERATION",  
       versions_starttime AS "START_DATE",  
       versions_endtime   AS "END_DATE", salary  
FROM employees  
   VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE  
WHERE  employee_id = 1;
```

no se han encontrado datos

FLASHBACK QUERY

- A continuación, creamos el empleado:

```
INSERT INTO copy_employees
VALUES (1, 'Natacha', 'Hansen', 'NHANSEN', '4412312341234',
       '07-SEP-1998', 'AD_VP', 12000, null, 100, 90, NULL);
```

```
SELECT employee_id, first_name || ' ' || last_name AS "NAME",
       versions_operation AS "OPERATION",
       versions_starttime AS "START_DATE",
       versions_endtime AS "END_DATE", salary
FROM employees
     VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE
WHERE employee_id = 1;
```

EMPLOYEE_ID	NAME	OPERATION	START_DATE	END_DATE	SALARY
1	Natacha Hansen	I	07-SEP-1998 06.51.58 AM	-	12000

FLASHBACK QUERY

- A continuación, puede actualizar la fila:

```
UPDATE copy_employees  
SET salary = 1  
WHERE employee_id = 1;
```

```
SELECT employee_id,first_name ||' '|| last_name AS "NAME",  
       versions_operation AS "OPERATION",  
       versions_starttime AS "START_DATE",  
       versions_endtime   AS "END_DATE", salary  
FROM employees  
   VENSIONS BETWEEN SCN MINVALUE AND MAXVALUE  
WHERE  employee_id = 1;
```

EMPLOYEE_ID	NAME	OPERATION	START_DATE	END_DATE	SALARY
1	Natacha Hansen	U	07-SEP-1998 06.57.01 AM	-	1
1	Natacha Hansen	I	07-SEP-1998 06.51.58 AM	07-SEP-1998 06.57.01 AM	12000

FLASHBACK QUERY

- A continuación, puede suprimir la fila:

```
DELETE from copy_employees  
WHERE employee_id = 1;
```

```
SELECT employee_id,first_name ||' '|| last_name AS "NAME",  
       versions_operation AS "OPERATION",  
       versions_starttime AS "START_DATE",  
       versions_endtime  AS "END_DATE", salary  
FROM employees  
   VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE  
WHERE  employee_id = 1;
```

EMPLOYEE_ID	NAME	OPERATION	START_DATE	END_DATE	SALARY
1	Natacha Hansen	D	07-SEP-1998 07.00.10 AM	-	1
1	Natacha Hansen	U	07-SEP-1998 06.57.01 AM	07-SEP-1998 07.00.10 AM	1
1	Natacha Hansen	I	07-SEP-1998 06.51.58 AM	07-SEP-1998 06.57.01 AM	12000

FLASHBACK QUERY

- El resultado de la última consulta de la diapositiva anterior solo está disponible si se utiliza Flashback query, es decir, la cláusula VERSIONS
- Si intenta realizar una búsqueda normal de employee_id = 1 tras la sentencia delete, recibirá el error normal, No Data Found

```
SELECT employee_id, salary  
FROM copy_employees  
WHERE employee_id = 1;
```

LEALTAD

LA LEALTAD SE TRADUCE EN GARANTIZAR QUE, UNA VEZ ACEPTADA Y COMPROMETIDA UNA OPERACIÓN O UN DISEÑO, EL SISTEMA Y LAS POLÍTICAS ASOCIADAS SE MANTENGAN FIELES A ESA PROMESA A LO LARGO DEL TIEMPO.

**¡GRACIAS POR
SU ATENCIÓN!**



DUDAS

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES
CONSULTAR CUALQUIER DUDA QUE TE SURJA EN LA SEMANA